

Workshop 1 - 02bash

Parte 1. Bash avanzado: análisis de logs

```
feijo@Valenfei MINGW64 ~ (master)
$ cat > access.log <<'LOGEND'
2026-08-24 08:15:20 INFO user=ana action=login ip=192.168.1.10
2026-08-24 08:16:02 ERROR user=juan action=login ip=192.168.1.15
2026-08-24 08:16:18 INFO user=ana action=download ip=192.168.1.10
2026-08-24 08:17:33 WARN user=carlos action=login ip=192.168.1.22
2026-08-24 08:18:04 ERROR user=juan action=upload ip=192.168.1.15
2026-08-24 08:19:27 INFO user=ana action=logout ip=192.168.1.10
2026-08-24 08:20:11 ERROR user=juan action=login ip=192.168.1.15
LOGEND
feijo@Valenfei MINGW64 ~ (master)
$ |
```

A- Detectar errores

```
feijo@Valenfei MINGW64 ~ (master)
$ grep 'ERROR' access.log
2026-08-24 08:16:02 ERROR user=juan action=login ip=192.168.1.15
2026-08-24 08:18:04 ERROR user=juan action=upload ip=192.168.1.15
2026-08-24 08:20:11 ERROR user=juan action=login ip=192.168.1.15
feijo@Valenfei MINGW64 ~ (master)
$
```

B- Contar errores

```
feijo@Valenfei MINGW64 ~ (master)
$ grep 'ERROR' access.log | wc -l
3
```

C- Identificar usuarios con errores

```
feijo@Valenfei MINGW64 ~ (master)
$ grep 'ERROR' access.log | awk '{for(i=1;i<=NF;i++) if($i ~ /^user=/) print $i}'
user=juan
user=juan
user=juan
```

D- Contar errores por usuario

```
feijo@Valenfei MINGW64 ~ (master)
$ grep 'ERROR' access.log | awk '{for(i=1;i<=NF;i++) if($i ~ /\user=/) print $i}' | sort | uniq -c
3 user=juan
```

E- Identificar la IP más asociada a errores

```
feijo@Valenfei MINGW64 ~ (master)
$ grep "ERROR" access.log | grep -oE "ip=[0-9.]+\" | cut -d'=' -f2 | sort | uniq -c | sort -nr
3 192.168.1.15
```

F-Generar un reporte

```
MINGW64:/c/Users/feijo

feijo@Valenfei MINGW64 ~ (master)
$ echo "=== REPORTE DE ANÁLISIS DE LOGS ===" > reporte.txt

feijo@Valenfei MINGW64 ~ (master)
$ echo -n "1. Cantidad total de registros: " >> reporte.txt
wc -l < access.log >> reporte.txt

feijo@Valenfei MINGW64 ~ (master)
$ echo -n "2. Cantidad de errores (ERROR): " >> reporte.txt
grep -c "ERROR" access.log >> reporte.txt

feijo@Valenfei MINGW64 ~ (master)
$ echo -n "3. Cantidad de advertencias (WARN): " >> reporte.txt
grep -c "WARN" access.log >> reporte.txt

feijo@Valenfei MINGW64 ~ (master)
$ echo -n "4. Usuario con más errores: " >> reporte.txt
grep 'ERROR' access.log | awk '{for(i=1;i<=NF;i++) if($i ~ /\user=/) {split($i, a, "="); print a[2]}}' | sort |

feijo@Valenfei MINGW64 ~ (master)
$
```

```
feijo@Valenfei MINGW64 ~ (master)
$ echo -n "5. IP con más errores: " >> reporte.txt
grep "ERROR" access.log | grep -oE "ip=[0-9.]+\" | cut -d'=' -f2 | sort | uniq -c | sort -nr | head -n1 >
> reporte.txt

feijo@Valenfei MINGW64 ~ (master)
$ cat reporte.txt
=== REPORTE DE ANÁLISIS DE LOGS ===
1. Cantidad total de registros: 7
2. Cantidad de errores (ERROR): 3
3. Cantidad de advertencias (WARN): 1
4. Usuario con más errores:      3 juan
5. IP con más errores:         3 192.168.1.15
```

Desafío

```
feijo@Valenfei MINGW64 ~/02-bash (master)
$ grep 'ERROR' access.log | awk '{for(i=1;i<=NF;i++) if($i ~ /\user=/) {split($i, a, "="); print a[2]}}' | sort |
uniq -c | awk '$1 >= 2 {print "!!! ALERTA SOSPECHOSA: El usuario \"047\" $2 \"047 tiene \" $1 \" errores registrados
.\"}'
!!! ALERTA SOSPECHOSA: El usuario 'juan' tiene 3 errores registrados.

feijo@Valenfei MINGW64 ~/02-bash (master)
$
```

Parte 2. Ejecutar archivos Bash

```

feijo@Valenfei MINGW64 ~/02-bash (master)
$ echo '#!/bin/bash' > saludo.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ echo 'echo "Hola desde mi primer script Bash"' >> saludo.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ echo 'echo "Usuario: $(whoami)"' >> saludo.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ echo 'echo "Directorio actual: $(pwd)"' >> saludo.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ cat saludo.sh
#!/bin/bash
echo "Hola desde mi primer script Bash"
echo "Usuario: $(whoami)"
echo "Directorio actual: $(pwd)"

feijo@Valenfei MINGW64 ~/02-bash (master)
$ bash saludo.sh
Hola desde mi primer script Bash
Usuario: feijo
Directorio actual: /c/Users/feijo/02-bash

feijo@Valenfei MINGW64 ~/02-bash (master)
$ |

```

```

feijo@Valenfei MINGW64 ~/02-bash (master)
$ chmod +x saludo.sh
./saludo.sh
Hola desde mi primer script Bash
Usuario: feijo
Directorio actual: /c/Users/feijo/02-bash

```

Preguntas:

1. ¿Qué función cumple #!/bin/bash?

Su función es indicarle al kernel del sistema operativo qué intérprete de comandos (en este caso, la ruta absoluta de Bash) debe iniciar para procesar el código de este archivo cuando se ejecuta directamente

2. ¿Qué diferencia existe entre `bash saludo.sh` y `./saludo.sh`?

- `bash saludo.sh`: Llama al programa `bash` de forma explícita y le pasa el archivo como entrada.
- `./saludo.sh`: Intenta ejecutar el archivo de forma directa e independiente.

3. ¿Qué ocurre si el archivo no tiene permisos de ejecución?

El sistema operativo bloquea la ejecución y arroja un error de **"Permiso denegado"** (*Permission denied*) al intentar correrlo de forma directa con `./`

4. ¿Qué información devuelve \$? después de ejecutar un comando?

Devuelve el **código de salida** (*exit status*) del último comando ejecutado.

Parte 3. Ejercicio: script sin argumentos

```
$ cat << 'EOF' > monitor.sh
#!/bin/bash

echo "=====
echo "          MONITOR BÁSICO DEL CONTENEDOR"
echo "=====

echo "1. Hostname: $(hostname)"
echo "2. Usuario actual: $(whoami)"
echo "3. Fecha y hora: $(date)"
echo "4. Directorio actual: $(pwd)"

echo "5. Espacio en disco:"
df -h
echo ""

echo -n "6. Cantidad aproximada de procesos activos: "
ps -e | wc -l

echo "===== FIN DEL REPORTE ====="
EOF

feijo@Valenfei MINGW64 ~/02-bash (master)
$ chmod +x monitor.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ ./monitor.sh
=====
          MONITOR BÁSICO DEL CONTENEDOR
=====
1. Hostname: Valenfei
2. Usuario actual: feijo
3. Fecha y hora: Sat Aug 29 22:58:14 HPS 2026
4. Directorio actual: /c/Users/feijo/02-bash
5. Espacio en disco:
Filesystem      Size  Used Avail Use% Mounted on
C:/Program Files/Git  475G  470G   5.2G  99% /
6. Cantidad aproximada de procesos activos: 6
===== FIN DEL REPORTE =====

feijo@Valenfei MINGW64 ~/02-bash (master)
$
```

Parte 4. Ejercicio: script con argumentos

```

feijo@Valenfei MINGW64 ~/02-bash (master)
$ cat << 'EOF' > analiza.sh
#!/bin/bash

if [ $# -lt 1 ]; then
    echo "Error: No se especifico ningun archivo."
    echo "Uso correcto: $0 <archivo1> [archivo2 ...]"
    exit 1
fi

(EOF)
for ARCHIVO in "$@"; do
    echo "=====
    echo "INFORMACION DE ARCHIVO: $ARCHIVO"
    echo "=====

    if [ ! -f "$ARCHIVO" ]; then
        echo "Error: El archivo '$ARCHIVO' no existe."
        echo "=====
        echo ""
        continue
    fi

    echo "Archivo: $ARCHIVO"

    LINEAS=$(wc -l < "$ARCHIVO")
    echo "- Cantidad de lineas: $LINEAS"

    PALABRAS=$(wc -w < "$ARCHIVO")
    echo "- Cantidad de palabras: $PALABRAS"

    CARACTERES=$(wc -m < "$ARCHIVO")
    echo "- Cantidad de caracteres: $CARACTERES"

    ERRORES=$(grep -c "ERROR" "$ARCHIVO" 2>/dev/null)
    echo "- Lineas con ERROR: $ERRORES"

    ADVERTENCIAS=$(grep -c "WARN" "$ARCHIVO" 2>/dev/null)
    echo "- Lineas con WARN: $ADVERTENCIAS"

    echo "=====
    echo ""
done
EOF
feijo@Valenfei MINGW64 ~/02-bash (master)
$

```

```

feijo@Valenfei MINGW64 ~/02-bash (master)
$ chmod +x analiza.sh

feijo@Valenfei MINGW64 ~/02-bash (master)
$ ./analiza.sh
Error: No se especifico ningun archivo.
Uso correcto: ./analiza.sh <archivo1> [archivo2 ...]

feijo@Valenfei MINGW64 ~/02-bash (master)
$ ./analiza.sh <archivo1>
bash: syntax error near unexpected token `newline'

feijo@Valenfei MINGW64 ~/02-bash (master)
$ ./analiza.sh access.log
./analiza.sh: line 9: access.log: command not found
=====
INFORMACION DE ARCHIVO: access.log
=====
Archivo: access.log
- Cantidad de lineas: 7
- Cantidad de palabras: 42
- Cantidad de caracteres: 455
- Lineas con ERROR: 3
- Lineas con WARN: 1
=====

```